

# 哲学定位

本技能是一个**一次性安装程序**，不是对话式 Agent。运行时生成所有护栏配置，然后退出。

护栏本身是**沉默的防御系统**——它们通过 Git pre-commit、Maven 插件、GitHub Actions 自动执行，不需要人来对话触发。只有当代码触碰红线（编译失败、高危漏洞、`javax.*` 污染、缺少文档），它们才会熔断拦截。

```
你运行 /guardrails-java（安装，一次性的）
      ↓
护栏沉默运行（每次 commit / push / CI 自动触发）
      ↓
触碰红线 → 熔断拦截 → 拒绝提交/合并
```

## workflow 总览

项目检测（Maven/Gradle? Java 版本? Spring Boot 版本? Spring Boot 2.x 还是 3.x?）

- 模块 1: Pre-commit 本地熔断脚本
- 模块 2: 安全扫描（OWASP Dependency Check）
- 模块 3: API 文档自动生成（Springdoc OpenAPI）
- 模块 4: Docker 容器化（Dockerfile + docker-compose-env.yml）
- 模块 5: GitHub Actions CI/CD 流水线
- 模块 6: K8s 部署清单（按需）

每个模块独立可用。

## 模块 1：Pre-commit 本地熔断脚本

### 1.1 生成 `.git/hooks/pre-commit`

写入以下脚本并 `chmod +x`：

```
#!/bin/sh
echo "===== 【工程护栏】 代码与安全合规性扫描 ====="

# 1. 编译与单元测试大闸
echo "> 正在运行单元测试与编译检查..."
mvn clean test
if [ $? -ne 0 ]; then
    echo "❌ [ERROR] 单元测试未通过或编译失败！拒绝提交。"
    exit 1
fi

# 2. Jakarta 命名空间污染检测（Spring Boot 3 核心大闸）
if grep -q "spring-boot-starter-parent" pom.xml 2>/dev/null; then
    BOOT_VERSION=$(grep -A1 "spring-boot-starter-parent" pom.xml | grep "
<version>" | sed 's/.*<version>\(.*\)</version>.*\/1/')
    case "$BOOT_VERSION" in
        3.*)
```

```

        if git diff --cached --name-only | xargs grep -l "import
javax\.servlet\|import javax\.annotation\|import javax\.persistence" 2>/dev/null;
then

        echo "❌ [CRITICAL RISK] Spring Boot 3.x 项目检测到 javax.* 命名空
间! "

        echo "    请替换为 jakarta.* 对应包: "
        echo "    javax.servlet.* → jakarta.servlet.*"
        echo "    javax.annotation.* → jakarta.annotation.*"
        echo "    javax.persistence.* → jakarta.persistence.*"
        echo "    同时检查: Swagger 2.x 需替换为 SpringDoc OpenAPI"
        exit 1

    fi

;;

esac
fi

# 3. 核心文档追溯检查
if [ ! -f "CONTEXT.md" ]; then
    echo "❌ [ERROR] 项目缺少核心上下文文件 CONTEXT.md! "
    echo "    请运行 /plan-java 初始化项目上下文, 或手动创建 CONTEXT.md。"
    exit 1
fi

echo "✅ [SUCCESS] 所有工程护栏通过, 允许提交! "
exit 0

```

## 1.2 自动检测构建工具

- 检测到 `pom.xml` → 使用 `mvn`
- 检测到 `build.gradle` / `build.gradle.kts` → 替换 `mvn clean test` 为 `./gradlew test`
- 如果同时存在, 优先 Maven

## 1.3 兼容现有 Git Guardrails

如果用户已安装 `git-guardrails-claude-code` (Claude Code 的 PreToolUse 钩子), 本 pre-commit 脚本与它互补:

- **Claude Code PreToolUse**: 拦截 Claude 执行 `git push --force`、`git reset --hard` 等危险命令
- **本 pre-commit**: 拦截编译失败、`javax.*` 污染、缺少文档的提交

两层不冲突, 建议都装。

# 模块 2: 安全扫描

## 2.1 OWASP Dependency Check (Maven)

在 `pom.xml` 的 `<build><plugins>` 中追加:

```

<plugin>
  <groupId>org.owasp</groupId>
  <artifactId>dependency-check-maven</artifactId>

```

```

<version>10.0.4</version>
<configuration>
  <!-- CVSS ≥ 7 (High 及以上) 阻断构建 -->
  <failBuildOnCVSS>7</failBuildOnCVSS>
  <formats>
    <format>HTML</format>
    <format>JSON</format>
  </formats>
</configuration>
<executions>
  <execution>
    <goals>
      <goal>check</goal>
    </goals>
  </execution>
</executions>
</plugin>

```

规则：

- 每次 `mvn verify` 自动扫描全部依赖
- 发现已知 CVE 漏洞且 CVSS ≥ 7 → 构建中断
- 首次运行下载 CVE 数据库，耗时 5-10 分钟

## 2.2 Trivy 镜像扫描 (CI 中追加)

在 CI 的 Docker 构建步骤后追加：

```

- name: Build Docker image
  run: docker build -t app:${{ github.sha }} .

- name: Scan image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: app:${{ github.sha }}
    format: 'table'
    exit-code: '1'
    severity: 'CRITICAL,HIGH'

```

两层安全覆盖：OWASP 扫 JAR 依赖 → Trivy 扫容器镜像。

# 模块 3：API 文档自动生成 (Springdoc OpenAPI)

## 3.1 检测与配置

如果项目是 Spring Boot 3.x (从 `pom.xml` 中 `spring-boot-starter-parent` 版本判断)：

提醒用户添加 Springdoc 依赖。不手动写 Swagger 注解，代码即文档：

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

如果项目是 Spring Boot 2.x:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-ui</artifactId>
  <version>1.8.0</version>
</dependency>
```

## 3.2 配置 (application.yml)

```
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
  default-produces-media-type: application/json
```

## 3.3 效果

- 项目启动后, 访问 `http://localhost:8080/swagger-ui.html` 即可看到自动生成的 API 文档
- Controller 的 `@RestController`、`@GetMapping`、`@RequestParam` 等标准注解会自动生成 OpenAPI 3.0 文档
- Javadoc 注释中的 `@param`、`@return` 会被提取到文档中

**核心原则:** 拒绝手动维护文档。代码改了什么, 文档自动同步。

# 模块 4: Docker 容器化

## 4.1 Dockerfile (多阶段构建)

根据项目 Java 版本自动选择基础镜像:

```
# 阶段 1: 构建
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn package -DskipTests -B

# 阶段 2: 运行 - 最小 JRE 镜像
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
```

```
ENV JAVA_OPTS="-Xms256m -Xmx512m -XX:+UseG1GC -XX:MaxGCPauseMillis=200"

EXPOSE 8080

ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

版本匹配规则: Java 17 → `eclipse-temurin:17-*`; Java 21 → `eclipse-temurin:21-*`; Java 11 → `eclipse-temurin:11-jre`; Java 8 → `openjdk:8-jre-alpine`。

## 4.2 docker-compose-env.yml (本地开发中间件底座)

只生成项目实际用到的中间件:

```
version: '3.8'

services:
  postgres-db:
    image: postgres:15-alpine
    container_name: enterprise-postgres
    environment:
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: app_password
      POSTGRES_DB: enterprise_core
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis-cache:
    image: redis:7-alpine
    container_name: enterprise-redis
    command: redis-server --requirepass app_redis_password
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

联动逻辑: `/tdd-java` 在执行集成测试时, 会自动 `docker-compose -f docker-compose-env.yml up -d` 拉起中间件环境, 测试完成后销毁。这要求提醒用户在 `application-test.yml` 中配置对应的连接信息。

## 模块 5: GitHub Actions CI/CD 流水线

### 5.1 自动检测

| 检测到                  | 构建工具  | 编译命令                     | 测试命令                  |
|----------------------|-------|--------------------------|-----------------------|
| <code>pom.xml</code> | Maven | <code>mvn compile</code> | <code>mvn test</code> |

| 检测到                                                          | 构建工具   | 编译命令                                               | 测试命令                                        |
|--------------------------------------------------------------|--------|----------------------------------------------------|---------------------------------------------|
| <code>build.gradle /</code><br><code>build.gradle.kts</code> | Gradle | <code>./gradlew</code><br><code>compileJava</code> | <code>./gradlew</code><br><code>test</code> |

从 `pom.xml` 的 `<properties>` 或 `build.gradle` 的 `sourceCompatibility` 提取 Java 版本。

## 5.2 流水线生成

在 `.github/workflows/ci.yml` 生成:

```
name: CI

on:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java-version: [{检测到的 Java 版本}]

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: testdb
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4
```

```

- name: Set up JDK {版本}
  uses: actions/setup-java@v4
  with:
    java-version: '{版本}'
    distribution: 'temurin'
    cache: '{maven | gradle}'

- name: Compile
  run: {mvn compile | ./gradlew compileJava}

- name: Test
  run: {mvn test | ./gradlew test}

- name: Dependency Check (OWASP)
  run: mvn dependency-check:check -DskipTests
  continue-on-error: true

- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: target/surefire-reports/

- name: Upload dependency check report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: dependency-check-report
    path: target/dependency-check-report.html

```

#### 智能检测逻辑:

- Postgres 依赖 → 自动加入 postgres service
- Redis 依赖 → 自动加入 redis service
- 消息队列依赖 → 不加 CI service, 提示用 Testcontainers
- 多模块项目 → 用 `mvn test -pl {module}` 代替全量测试

## 模块 6: K8s 部署清单 (按需)

只在用户明确需要时生成。创建 `k8s/` 目录:

### 6.1 Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {app-name}
  labels:
    app: {app-name}
spec:
  replicas: 2

```

```

selector:
  matchLabels:
    app: {app-name}
template:
  metadata:
    labels:
      app: {app-name}
  spec:
    containers:
      - name: {app-name}
        image: {registry}/{app-name}:latest
        ports:
          - containerPort: 8080
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: "k8s"
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
            initialDelaySeconds: 10
            periodSeconds: 5

```

## 6.2 Service

```

apiVersion: v1
kind: Service
metadata:
  name: {app-name}
spec:
  selector:
    app: {app-name}
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP

```

**前置条件:** 需要 `spring-boot-starter-actuator` 依赖。如果没有，提醒用户添加。

# 交互约束

| 步骤         | 模板                                                     |
|------------|--------------------------------------------------------|
| 项目检测       | 检测到: Maven, Java {版本}, Spring Boot {版本}。               |
| Pre-commit | .git/hooks/pre-commit 已安装, 含编译 + Jakarta 检测 + 文档检测。    |
| 安全扫描       | OWASP Dependency Check 已配置 (CVSS ≥ 7 阻断) + Trivy 镜像扫描。 |
| 文档生成       | Springdoc OpenAPI 已配置, 启动后访问 /swagger-ui.html。         |
| Docker     | Dockerfile + docker-compose-env.yml 已生成。               |
| CI/CD      | .github/workflows/ci.yml 已生成。                          |
| K8s        | k8s/deployment.yaml + service.yaml 已生成。                |

# 全局 workflow 全景

